

Речь к презентации AttendPro v4 (BLE-first)

Файл презентации: [AttendPro_v4_BLE_Architecture.pptx](#) · 19 слайдов **Хронометраж:** ~15–18 минут + вопросы **Аудитория:** руководитель

Совет по подаче: не читать дословно — это опорный текст. Жирным выделены фразы-якоря, которые стоит произнести обязательно.

Слайд 1 — Титул

Здравствуйте! Это Мария, и сегодня я расскажу про AttendPro — систему отметки посещаемости занятий, которую мы проектируем. В четвёртой версии архитектуры мы убираем QR-коды и переходим на Bluetooth — и за следующие пятнадцать минут я покажу, почему это не «модно», а единственный способ решить главную проблему: **недоказуемость времени в офлайне**. План такой: сначала проблема и выбор технологии, потом — как устроена сама отметка, что за криптография летает по воздуху, где что хранится, и что мы честно не можем решить ничем радио. В конце — контейнерная архитектура и модель угроз.

Слайд 2 — Проблема: QR не может доказать время

Начну с боли. Сегодня студент сканирует QR — и всё, что система о нём знает, он рассказал сам. Перевёл часы назад — отметка «сделана вовремя». Сфотографировал код — отметится вечером из общежития. Можно спросить: а нельзя ли это детектировать? Мы провели глубокое исследование — и ответ фундаментальный: **нет**. Подпись доказывает авторство — «эти байты создал владелец этого ключа», — но не время. Если устройство никого доверенного не видело с момента пары до момента синхронизации, никакая база, блокчейн или криптография не отличит «создал в 10:05» от «создал в 12:00». Информации просто нет.

Справа — принцип решения: физическое присутствие становится обязательным, окно времени подписывает доверенное устройство преподавателя, а часы студента из формулы исчезают вообще. Плюс zero-trust: сервер никому на слово не верит и всё перепроверяет сам.

Слайд 3 — Обзор технологий

Как вообще передать данные «вблизи и без QR»? Мы перебрали весь зоопарк ближней связи. BLE — Bluetooth Low Energy: короткий радиообмен, месяцами живёт на батарейке — наш основной выбор. Wi-Fi LAN — запасной канал: та же железка хаба раздаёт локальную сеть без Интернета. NFC — касание на четыре сантиметра, значит очередь из двухсот человек. UWB и Channel Sounding — сверхточное расстояние и защита от ретрансляции, но таких телефонов пока мало. Wi-Fi Aware — поиск без роутера, но только на новых устройствах. BLE mesh — передача через чужие телефоны, что для нас фатально. LoRa — километры, но скорость в байты в секунду и своё железо у каждого. Ультразвук — записывается и проигрывается.

Выбираем BLE, с Wi-Fi LAN как запасным. Дальше три слайда — почему.

Слайд 4 — BLE: одна отметка — две фазы

Сразу сниму главный вопрос, который возникает здесь у всех. В BLE есть широковещательный маяк и соединение точка-точка — и это **не две сети, а два шага одной технологии**. Смотрите на баннер: фаза один — маяк находит пару, фаза два — соединение передаёт отметку. Убрать одну из фаз нельзя: без маяка телефон не найдёт хаб, без соединения не передать подпись.

Немного физики, одним абзацем. BLE живёт на волнах 2,4 гигагерца — это «общий эфир», не требующий лицензий; дальность — единицы-десятки метров, стены и тела сигнал ослабляют. В диапазоне сорок каналов, три из них — рекламные, выбраны в щелях между Wi-Fi. Маяк — это крошечный пакет в 31 байт, который хаб выкрикивает раз в секунду без всякого соединения; энергии это стоит копейки, и слушать почти бесплатно. На схеме справа — фаза один: хаб кричит, телефоны слушают. Так работали маячки магазинов iBeacon и система Exposure Notification от Apple и Google. Важно: **услышать маяк может каждый — но услышать не значит отметить**. Отметка рождается только в фазе два, на следующем слайде.

Слайд 5 — GATT: разговор точка-точка

Фаза два — GATT, Generic Attribute Profile, режим настоящего двустороннего соединения. Здесь две роли. Peripheral, «периферия», — маленький сервер: он рекламирует себя и ждёт подключения. Central, «центр», — клиент: он услышал маяк, подключился, читает и пишет. Внутри соединения — характеристики, ячейки с тремя операциями: прочитать, записать, подписаться на ответ.

Наше распределение ролей видно на схеме справа: хаб — peripheral с тремя ячейками: challenge — что читает студент, mark — куда пишет свою отметку, receipt — по которой приходит квитанция. Студент — central. Итог всей истории с фазами: маяк, подключение, обмен, отключение — **десять-тридцать секунд радио на одну отметку**. Больше телефону и не нужно.

Слайд 6 — Capacitor

Следующий практический вопрос: в чём это всё бегаёт? У нас веб-приложение — и тут упираемся в платформу: браузерный Web Bluetooth не существует в Safari, то есть **ни один iPhone не умеет BLE из веб-страницы**, это ~76 процентов поддержки только в браузерах на Chromium. Решение — Capacitor: упаковщик, который берёт наш веб-код и заворачивает его в нативное приложение для iOS и Android. Смотрите на слои справа: нативная оболочка даёт разрешения операционной системы и BLE-плагин, в WebView крутится наш же веб-код с UI и логикой, а внизу — Giper Baza в IndexedDB, офлайн-хранилище доказательств. Плагин-мост — @capacitor-community/bluetooth-le — умеет роль central: сканировать, подключаться, читать и писать. И то, что он не умеет peripheral — рекламироваться — нас не беспокоит: рекламироваться должен только хаб, студентам хватает клиента. Один код — три оболочки: iOS, Android и вкладка браузера. И практическая деталь для пилота: **публиковаться в App Store не обязательно** — iOS-сборку раздаём через **TestFlight**: до ста внутренних тестеров вообще без ревью, до десяти тысяч внешних — с упрощённой бета-проверкой, несравнимо легче релизной. На Android ещё проще — ставим APK-файл напрямую.

Слайд 7 — Сводная таблица выбора

Собираю выбор в одну таблицу. BLE плюс Capacitor-плагин — выбор: надёжно на обеих платформах и даёт доказательство близости. Wi-Fi LAN — запасной: десятки HTTP-отметок в секунду, но подключение к сети — ручной шаг, и фактор близости всё равно даёт BLE-маяк. Web Bluetooth — нет: в Safari его нет вообще. NFC — нет: очередь. UWB и Channel Sounding — позже: это

криптозащита от relay-атак, но парк устройств пока мал. Wi-Fi Aware — позже, BLE mesh — нет: ретрансляция чужими телефонами уничтожает доказательство аудитории. LoRa — нет, не для комнаты. Одна строка внизу — важная оговорка: Wi-Fi LAN у нас именно запасной, а не второй основной.

Слайд 8 — Стратегия MVP → Target

Теперь вопрос стратегии: с чего начинаем пилот? В таблице три пути. Первый: MVP на онлайн-QR, потом офлайн-BLE. Второй: MVP на онлайн-BLE. Третий: сразу офлайн-BLE, он же Target.

Пройдём по критериям. Время в онлайн безопасно везде — сервер штампует приём. А вот присутствие QR не доказывает и в онлайн: скриншот или просто ссылка уезжает другу домой. Путь два — это онлайн-BLE **без Гипер Базы**: быстрый старт — простое хранилище и REST, радио и подписанные факты в нём сохраняются и переедут в Target без потерь. Но при выходе в офлайн слой хранения и синхронизации придётся переписывать на Baza — REST-сервер не умеет «доехать позже». **Выбираем путь три**: Giper Baza с первого дня — тогда онлайн достаётся бесплатно (хаб с сетью синкает мгновенно), офлайн работает на том же коде, а QR остаётся аварийным онлайн-фолбэком для сломанного Bluetooth или гостя без приложения.

Слайд 9 — Ключевая идея: witness

Сердце системы. Кто доказывает время? **Хаб — свидетель**. Схема сверху читается слева направо: хаб открывает окно и подписывает якорь сессии; студент отвечает своей подписанной отметкой; хаб возвращает квитанцию — «получил, пока окно было открыто». Отметьте себе: это две половины доказательства. Студент доказывает авторство, хаб — своевременность. Поодиночке они не стоят ничего: отметка без квитанции — «поверил на слово», квитанция без отметки — ничья.

Как устроено окно: якорь перевыпускается каждые несколько минут, цепочкой хэшей — это, по сути, QR-ротация, только по радио. Поля времени в отметке **нет вообще** — оно выводится из окна якоря. В конце пары хаб подписывает WitnessBatch — Merkle-корень всех принятых отметок. И mesh у нас запрещён: только один прямой радио-хоп, потому что подпись доказывает авторство, а не близость.

Слайд 10 — Радиопротокол

Теперь как это выглядит в эфире. **Кричит только хаб**. Один рекламодатель и сотни слушающих — самая лёгкая из всех радио-конфигураций, коллизий толпы просто не возникает. Студент подключается короткой сессией: connect, отметить, disconnect — хаб не держит сотни линий. Якорь ротируется эпохами, опоздавшие отмечаются в поздних эпохах тем же флоу. Фоновые ограничения операционки мы обошли архитектурно: one-tap в foreground, когда приложение открыто, ограничений нет. Напоминание — опция по расписанию, без геолокации: расписание уже скачано, телефон знает «сегодня в десять пара», и в десять ноль две приходит локальное уведомление.

Слайд 11 — Платформы и ёмкость

Технические характеристики, честно. iPhone: one-tap в открытом приложении — надёжно, без ограничений. В фоне iOS режет и скан, и рекламу — и это не лечится даже самим Apple: в Exposure

Notification iPhone вещал только при включённом экране. Поэтому one-tap у нас основной режим, и поэтому нужен Capacitor. Android: то же самое плюс опциональный фоновый скан по расписанию; капризы агрессивной экономилки батареи у Xiaomi учитываем в матрице пилота.

Ёмкость: сотни слушателей одного маяка — без конфликтов; сто-триста отметок проходят за две-пять минут. Теперь каверзные сценарии — по каждому объясню, что физически происходит и как лечим.

Толпа ломится сразу, или «громовое стадо». Представьте: сто человек тапнули «Отметиться» в одну секунду. У хаба слотов одновременных подключений мало — единицы. Без защиты запросы на соединение сталкиваются в эфире, часть клиентов получает отказ — и, что хуже всего, все отказавшиеся повторяют попытку снова одновременно, снова толпой, и поток захлёбывается волной повторов. Лечение двухступенчатое. Первое — **джиттер**: случайная пауза перед подключением, каждый клиент ждёт своё случайное время от нуля до пары секунд; толпа автоматически растягивается в очередь, как люди, расходящиеся по времени в двери, а не в один миг. Второе — **экспоненциальный откат**: каждая неудачная попытка удваивает ожидание повтора (с новой случайной добавкой), поэтому неудачники разносятся во времени автоматически и поток сам выравнивается. Сессия у каждого короткая — секунды, — поэтому очередь из трёхсот человек проходит за минуты, а Wi-Fi LAN вообще снимает пиковую нагрузку.

Карман и стена. Маяк — это не одно сообщение, а повторяющийся поток: пакет ежесекундно, тридцать независимых попыток за полминуты. Телефону в кармане или за бетонной колонной достаточно поймать **один** пакет из тридцати — сигнал телом и тканью ослабляется, но не гасится. Если совсем глухо — студент достаёт телефон, это же one-tap-флоу.

Две пары рядом. В маяке лежит идентификатор сессии — `session_handle`, и скан приложения фильтруется по нему: чужой маяк из соседней аудитории интерфейс даже не разбудит. Ротация эпох дополнительно убивает устаревшие идентификаторы.

Батарея. Единственная дорогая операция — сканирование, но у нас оно длится секунды в момент отметки, и студенты вообще ничего не рекламируют. Для сравнения: Exposure Notification держал и скан, и рекламу постоянно на всём парке телефонов мира — наш профиль в десятки раз легче.

Слайд 12 — Флоу: отметка за 7 шагов

Сам флоу, слева вниз по стрелкам. Студент открывает приложение — или тапает напоминание. Скан две-пять секунд: «Пара Алгоритмы, аудитория 312». Тап «Отметиться» — подключение с джиттером, если толпа. Читает якорь, проверяет подпись хаба и свой креншел — это единственные локальные проверки. Создаёт Mark в своей Giper Baza, подписывает. Пишет в характеристику; хаб проверяет подпись и допуск. Квитанция назад — «принято преподавателем» — и отключение.

Справа инвариант интерфейса, который мы держим железно: «найдено», «сохранено локально», «принято хабом». Зелёная галочка появляется **только** после квитанции. Опоздавшие — та же схема с текущей эпохой. В конце пары — WitnessBatch, и дальше синхронизация, когда появится любая сеть.

Слайд 13 — Подписанные факты

Что конкретно летает по воздуху — четыре факта. SessionAnchor от хаба: идентификатор пары, номер эпохи, случайный нонс на 128 бит, хэш предыдущего якоря, отпечаток ключа. Mark от студента: хэш якоря, хэш своего credenшела, нонс — и заметьте, **поля времени нет**. Receipt от хаба: хэш отметки, хэш якоря, монотонный номер приёма. WitnessBatch при закрытии: Merkle-корень всех принятых отметок.

Все факты — каноническая сериализация, хэш которой и есть идентификатор факта; отдельные uuid не заводим. Плюс три факта онбординга и вердиктов: credenшелы студента и преподавателя подписывает SSO-валидатор, решения — синхронизатор.

Слайд 14 — Хаб преподавателя: три варианта

На чём это крутится у преподавателя — три варианта радио. А — встроенный Bluetooth ноутбука с маленькой нативной прослойкой: работает, но качество стека Windows — легенда, и это вариант для пилота. Б — ESP32-свисток в USB за пять-восемь долларов: вся радиочасть на платке, ноут говорит с ней как с модемом; радио гарантировано на любом железе, ключ можно держать в защищённом железе чипа, и ту же платку потом ставим стационарно в аудиторию — **рекомендуемый вариант**. В — телефон преподавателя: возможен, но требует кастомного нативного плагина, оставляем как фолбэк. Внизу — Wi-Fi LAN: та же железка раздаёт сеть без Интернета, те же подписанные факты по HTTP.

Слайд 15 — Хранение: двойная запись

Где живут факты — и это сознательная двойная запись, не дублирование. Отметка создаётся у студента в его Giper Baza на телефоне — это его личные доказательства: своя подпись плюс квитанция хаба. Встречная копия и все квитанции — у хаба. Когда появляется любая сеть, дельта-синхронизация привозит всё синхронизатору, который zero-trust перепроверяет каждый факт и издаёт вердикты; дальше PostgreSQL и экспорт в Modeus.

Два следствия. Спор «почему мне не зачли» решается криптографией: у студента Mark и Receipt, у сервера Batch — ничьё слово не нужно. И страховочная дверь: конверты фактов самодостаточны, хранилище заменяемо без потери доказательности.

Слайд 16 — Почему Giper Baza

Почему в качестве движка Giper Baza — и честно, что мы у неё НЕ берём. Берём: подписанные неизменяемые юниты — целостность «на полке»; ленды с правами — персональные ленды студентов и офлайн-доставка решений; готовый дельта-синк — не пишем свой outbox; гибридные часы — бесплатный предохранитель; и один движок на браузере и Node — студент, хаб, синхронизатор.

Не берём: CRDT-слияние как таковое — у нас писатели не пересекаются, хватило бы union-логики, мы не обманываемся на этот счёт; шифрование лендов — прятать нечего, канал и так шифрует сам Bluetooth; и База не источник политик — она принимает всё подписанное, валидатор — наш код. Итог: берём не ради модного CRDT, а ради связки в одном пакете на нашем рантайме.

Слайд 17 — Акторы и авторизация

Кто действует в системе. Студент со своим ключом и credenшелом издаёт Mark. Преподаватель владеет ключом хаба и издаёт якоря, квитанции и батчи. SSO-валидатор — университетская identity-система: отдельный актор, потому что только она уполномочена связать ключ устройства с конкретным человеком из контингента; без неё «кто угодно отметил за кого угодно».

Синхронизатор — zero-trust верификатор, единственный, кто меняет официальную посещаемость. Да, вход по университетской учётке будет — при первом запуске SSO. Администратор — не отдельный актор: исправления оформляются подписанными фактами-переопределениями, история не переписывается. Итого четыре пломбы: университет, хаб, студент, хаб.

Слайд 18 — Контейнерная архитектура

Как это разложено по контейнерам. Слева фронт: студент — Capacitor-оболочка с WebView, нашим веб-кодом, Giper Baza в IndexedDB и BLE-плагином; хаб — веб-консоль, Hub Runtime на Node с WitnessLand, и Radio Bridge — ESP32 в USB либо нативный модуль. Между ними — BLE GATT и Wi-Fi LAN. Справа бек с жёстким разделением слоёв: платформа — узел Giper Baza с дельта-синком; сервисы — Ingress Validator с карантинном, Verifier Core с политикой и Identity Service с SSO; хранение — PostgreSQL как доказательное зеркало и адаптер Modeus. Онбординг идёт из Identity Service, а при недоступности Baza деградирует до онлайн-бека на тех же подписанных конвертах.

Слайд 19 — Модель угроз

Финал — честная граница безопасности. Что решено: перевод часов и запись в прошлое — окном якоря и квитанцией; отметка из дома — отсутствием радио-контакта; replay и дубли — нонсом эпохи и уникальностью отметки; поддельный хаб и Sybil-студент — цепочкой доверия SSO. Что не решено: друг принёс телефон студента — это уровень бумажной ведомости, закрывается выборочными визуальными проверками; и relay-туннель — радио-эстафета через сообщника. Криптографический ответ на него существует — Channel Sounding в Bluetooth 6.0 и UWB, измерение времени полёта сигнала, — но устройств с поддержкой пока мало, это фактор будущего. Отсюда главное правило внизу: подпись доказывает авторство, а не близость — поэтому mesh на пути отметки запрещён.

Спасибо! Готова к вопросам.

Возможные вопросы и заготовки ответов

— **Почему просто не вращающийся QR?** Вращение защищает от скриншота, но не от live-релея: экран стримится другу по видеозвонку в реальном времени. QR не даёт радио-касания — значит, не доказывает присутствие никогда, даже в онлайн.

— **А если у студента не работает Bluetooth / нет приложения?** Лестница деградации: BLE → Wi-Fi LAN того же хаба → provisional-отметка с ручным подтверждением преподавателем → аварийный онлайн-QR со статусом «требуется проверки». Правило одно: без свидетеля нет «принято».

— **Сколько это стоит в железе?** ESP32 — пять-восемь долларов на аудиторию, или ноль — ноутбук преподавателя в варианте А.

— **Что с приватностью?** В эфире только анонимный маяк сессии; личные данные идут по зашифрованному самому Bluetooth соединению; геолокацию не используем вообще; список у преподавателя можем отдать только счётчиком.

— **Когда можно потрогать?** MVP на двух устройствах: Mac-хаб + iPhone-студент; этапы — логика фактов, радио, Capacitor-сборка, затем Giper Baza и синк. Первая проверка — полигонные метрики: латентность скана, успешность цикла, пропускная способность.